

WEEK – 3

Java Programming – Java Buzzwords, Data Types, Variables, Operators, Control Structures, Arrays, Console Input and Output

PROGRAM 1

1. Title

Java Program to Check Whether the Given Number is Prime Palindrome or Not (CO1)

2. Objective / Aim

To write and execute a Java program to check whether the given number is both **prime** and **palindrome** using methods, loops, conditional statements, and console input.

3. Problem Statement

Write a Java program that reads an integer from the user and checks whether the number is a **Prime Palindrome**.

A number is a **Prime Palindrome** if:

- It is a prime number.
- It reads the same from left to right and right to left.

For example, 131 is both prime and palindrome.

4. Concepts Used

- Java data types
- Variables
- Operators
- `if-else` statements
- `for` loop

- `while` loop
 - Methods
 - Console input using `Scanner`
 - Modulus operator `%`
 - Integer division
-

5. Algorithm / Logic Description

To Check Prime

1. Read the number `n`.
2. If `n <= 1`, it is not prime.
3. Start a loop from 2 to `n / 2`.
4. Check whether `n` is divisible by the loop variable.
5. If divisible, return `false`.
6. Otherwise, return `true`.

To Check Palindrome

1. Store the original number.
2. Initialize `reverse = 0`.
3. Extract the last digit using `n % 10`.
4. Add the digit to the reverse number.
5. Remove the last digit using `n / 10`.
6. Repeat until the number becomes zero.
7. Compare the original number with the reversed number.
8. If both are equal, the number is a palindrome.

Final Check

9. Call both `isPrime()` and `isPalindrome()`.
 10. If both return `true`, display that the number is a Prime Palindrome.
 11. Otherwise, display that it is not a Prime Palindrome.
-

6. Source Code

```
package recordprograms;

import java.util.Scanner;

public class PrimePalindrome {

    // Method to check whether a number is prime
```

```

static boolean isPrime(int n) {
    if (n <= 1)
        return false;

    for (int i = 2; i <= n / 2; i++) {
        if (n % i == 0)
            return false;
    }

    return true;
}

// Method to check whether a number is palindrome
static boolean isPalindrome(int n) {
    int original = n;
    int reverse = 0;

    while (n > 0) {
        int digit = n % 10;
        reverse = reverse * 10 + digit;
        n = n / 10;
    }

    return original == reverse;
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter a number: ");
    int n = sc.nextInt();

    if (isPrime(n) && isPalindrome(n))
        System.out.println(n + " is a Prime Palindrome.");
    else
        System.out.println(n + " is not a Prime Palindrome.");

    sc.close();
}
}

```

7. Sample Output

Case 1: Prime Palindrome

```

Enter a number: 131
131 is a Prime Palindrome.

```

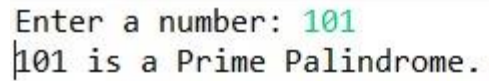
Case 2: Not a Prime Palindrome

```

Enter a number: 121
121 is not a Prime Palindrome.

```

```
Enter a number: 101
101 is a Prime Palindrome.
```



8. Result

The Java program was successfully implemented to check whether a given number is a **Prime Palindrome** using methods, conditional statements, loops, operators, and console input.

PROGRAM 2

1. Title

Java Program to Read Marks of Five Subjects Through Command Line and Print Total and Average (CO1)

2. Objective / Aim

To write and execute a Java program that accepts marks of five subjects through **command-line arguments** and calculates the total and average marks.

3. Problem Statement

Write a Java program that reads marks of five subjects through command-line arguments and displays:

- Total marks
- Average marks

The program should verify that exactly five marks are entered.

4. Concepts Used

- Command-line arguments
 - Arrays
 - Variables
 - Data types
 - `for` loop
 - Arithmetic operators
 - `Integer.parseInt()`
 - Console output
 - Conditional statements
-

5. Algorithm / Logic Description

1. Start the program.
 2. Check whether exactly five command-line arguments are provided.
 3. If the number of arguments is not five, display an error message and terminate.
 4. Initialize `total` to zero.
 5. Use a `for` loop to process the five marks.
 6. Convert each command-line argument from `String` to `int` using `Integer.parseInt()`.
 7. Add each mark to `total`.
 8. Calculate the average using `total / 5.0`.
 9. Display the total marks.
 10. Display the average marks.
 11. Stop the program.
-

6. Source Code

```
package recordprograms;

import java.util.Scanner;

public class Marks {

    public static void main(String[] args) {

        // Check whether exactly five marks are provided
        if (args.length != 5) {
```

```
        System.out.println("Please enter marks for exactly five
subjects.");
        return;
    }

    int total = 0;

    // Read and add the five marks
    for (int i = 0; i < 5; i++) {
        int mark = Integer.parseInt(args[i]);
        total = total + mark;
    }

    // Calculate average
    double average = total / 5.0;

    // Display result
    System.out.println("Total Marks = " + total);
    System.out.println("Average Marks = " + average);
}
}
```

7. Compilation and Execution

Compile the program:

```
javac recordprograms/Marks.java
```

Run the program by providing five marks as command-line arguments:

```
java recordprograms.Marks 85 90 78 92 88
```

8. Sample Output

```
Total Marks = 433
Average Marks = 86.6
```

Another Example

```
java recordprograms.Marks 75 80 85 90 70
```

Output:

```
Total Marks = 400
Average Marks = 80.0
```

```
<terminated> Marks [Java Application] C:\Users\anils-csm\Desktop\record\programs  
Total Marks = 425  
Average Marks = 85.0
```

9. Result

The Java program was successfully implemented to accept marks of five subjects through **command-line arguments** and calculate and display the **total and average marks**.

CONCLUSION

The programs successfully demonstrate fundamental Java programming concepts including **data types, variables, operators, conditional statements, loops, methods, arrays/command-line arguments, and console input and output**.
